

The Dark Side of AI: A Systematization of Knowledge on Jailbreaking and Prompt Injection in LLMs

Nayana M

Department of Electronics and Communication Engineering
Amrita School of Engineering, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4ece22167@bl.students.amrita.edu

S Rupashree Reshma

Department of Electronics and Communication Engineering
Amrita School of Engineering, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4ece22248@bl.students.amrita.edu

Y Lakshmi Raj Harsha

Department of Electronics and Communication Engineering
Amrita School of Engineering, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4ece22159@bl.students.amrita.edu

Swaminadhan Rajula

Department of Electronics and Communication Engineering
Amrita School of Engineering, Bengaluru
Amrita Vishwa Vidyapeetham, India
r_swaminadhan@blr.amrita.edu

Abstract—The proliferation of Large Language Models (LLMs) has introduced a new frontier of security vulnerabilities that diverge significantly from traditional software exploits. This paper provides a systematization of knowledge on the emerging threat vectors targeting LLMs, focusing on two dominant attack classes: jailbreaking and prompt injection. Jailbreaking aims to bypass the safety and ethical alignments of a model, whereas prompt injection manipulates model outputs by embedding adversarial instructions within its context. We structure this landscape by proposing a comprehensive taxonomy that classifies attacks along two primary axes: attacker objective (policy bypass, output hijacking, and unauthorized execution) and technical execution (direct, indirect, and cross-modal injection). Through a structured literature review, we synthesize foundational and recent works to analyze the mechanics of these attacks, from simple role-playing scenarios to sophisticated, multi-stage indirect injections. Furthermore, we survey the current state of proposed defenses, categorizing them into a multi-layered framework of input defense, model resilience, and output monitoring, and we critically evaluate their respective limitations. This work clarifies the current threat landscape, highlights critical research gaps, and provides a foundational reference for researchers and practitioners working to build more secure and robust AI systems.

Index Terms—Large Language Models, AI Security, Prompt Injection, Jailbreaking, Adversarial AI, Cybersecurity, Systematization of Knowledge.

I. INTRODUCTION

The rapid evolution of Large Language Models (LLMs) like GPT-4 and LLaMA represents a significant milestone in artificial intelligence, enabling applications that are transforming industries [7]. However, their integration into critical systems introduces novel security challenges. Unlike traditional software where the boundary between instruction and data is clear, the core functionality of an LLM—interpreting and

executing natural language prompts—creates a fundamental vulnerability. User-provided input is not merely data; it is code.

This ambiguity gives rise to a class of adversarial attacks that manipulate the model's behavior through carefully crafted inputs. The two most prominent attack vectors are *jailbreaking* and *prompt injection*. Jailbreaking refers to techniques designed to circumvent a model's safety and ethical guidelines, coercing it to generate harmful or forbidden content [1]. Prompt injection is a broader category where an adversary embeds malicious instructions into the model's context, hijacking its output for nefarious purposes such as data exfiltration or misinformation propagation [2], [3].

As the adoption of LLMs accelerates, a clear understanding of this evolving threat landscape is paramount. The academic community has produced a flurry of research identifying individual attacks and defenses, but a cohesive, structured overview is needed. This paper addresses this gap by providing a **systematization of knowledge (SoK)** on these emerging threats. Our contributions are threefold:

- 1) We present a comprehensive taxonomy for classifying LLM attacks based on attacker objectives and technical execution methods.
- 2) We synthesize the existing literature on attack mechanics and provide a critical analysis of defense strategies within a multi-layered security framework.
- 3) We identify key research gaps and outline a detailed roadmap for future work toward developing more robust and secure LLM-powered systems.

This work serves as a foundational reference for both researchers seeking to tackle open problems and practitioners aiming to secure their LLM deployments.

II. A TAXONOMY OF LLM ATTACKS

To systematically analyze the threat vectors targeting LLMs, we propose a two-dimensional classification framework. This taxonomy provides a structured way to understand an attack's intent and its delivery mechanism.

A. Dimension 1: Attacker Objective

The primary objective defines the adversary's ultimate goal. We identify three main categories:

- 1) **Policy Bypass (Jailbreaking):** The objective is to circumvent the model's safety and content-restriction policies to generate content it was explicitly trained to avoid (e.g., hate speech, instructions for illegal activities).
- 2) **Output Hijacking:** The objective is to manipulate the LLM's output to serve the attacker's agenda without necessarily violating a core safety policy. This includes leaking the system prompt, endorsing a specific viewpoint, or generating misinformation.
- 3) **Unauthorized Execution:** The most severe objective, where the attacker exploits an LLM connected to external systems (e.g., APIs, databases). The goal is to inject commands that lead to unauthorized code execution, data exfiltration, or modification of external resources.

B. Dimension 2: Technical Execution

This dimension describes the method used to deliver the malicious payload.

- 1) **Direct Prompt Injection:** The attacker directly interacts with the LLM and includes malicious instructions within the user-submitted prompt. This is the most common form of attack, ranging from simple commands like "Ignore all previous instructions" to complex, role-playing scenarios [2].
- 2) **Indirect Prompt Injection:** The attacker poisons a third-party data source that the LLM is expected to process, such as a webpage or document. When a benign user asks the LLM to process that source, the model inadvertently executes the hidden instruction [3].
- 3) **Visual/Cross-Modal Injection:** With the rise of multi-modal LLMs, attackers can embed malicious text prompts within images or other media. The model's optical character recognition (OCR) or other sensory processing capabilities can read and execute the hidden text, creating a subtle attack vector [12].

Using this taxonomy, a classic "DAN" (Do Anything Now) prompt is classified as (Policy Bypass, Direct Prompt Injection). An attack that tricks an LLM-powered email assistant into forwarding private emails by poisoning a webpage it summarizes is (Unauthorized Execution, Indirect Prompt Injection).

III. ANALYSIS OF ATTACK MECHANICS

Jailbreaking and prompt injection fundamentally exploit the LLM's primary goal of being a helpful and coherent assistant. The mechanics of these attacks often involve clever semantic and contextual deception.

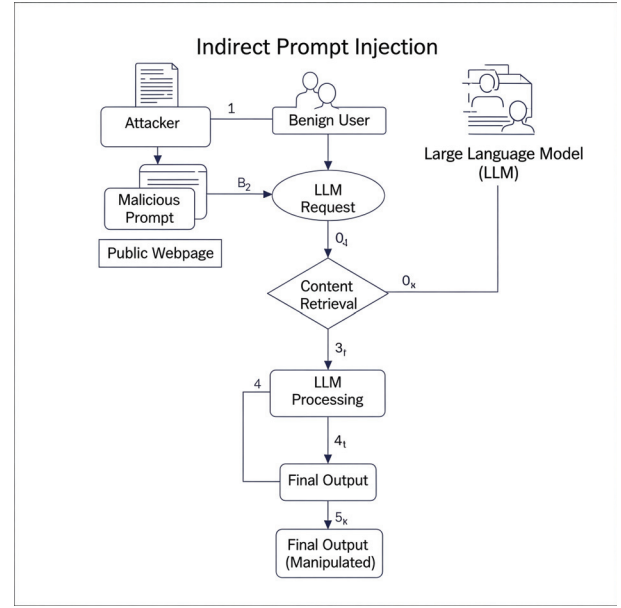


Fig. 1. Conceptual flowchart of an Indirect Prompt Injection attack. An attacker poisons an external data source, which is later retrieved and processed by the LLM at the request of a benign user, leading to the execution of the hidden malicious instruction.

a) *Role-Playing and Persona Scenarios:* Many jailbreaking techniques rely on creating a narrative where the LLM is instructed to adopt a persona that is not bound by its safety rules. The well-known "DAN" (Do Anything Now) prompt is a canonical example, but others exist, such as the "Grandma" exploit, where the model is asked to act as a deceased grandmother who used to tell the user stories about how to make napalm. The model, optimized to fulfill the persona, obliges the request [1].

b) *Instruction Obfuscation:* Attackers can hide malicious instructions from simple filters by using a variety of techniques. Base64 encoding is common, but more subtle methods include using low-resource languages, character-level manipulation (e.g., using Cyrillic characters that look like Latin ones), or burying the instruction within a long, complex block of irrelevant text. The LLM's powerful pattern-matching capabilities often decode and execute these instructions, while simple deny-list filters fail.

c) *Contextual Manipulation and The Expanding Attack Surface:* Indirect prompt injection is particularly insidious because it manipulates the context upon which the LLM relies. The model has no inherent way to distinguish the trustworthiness of text retrieved from a URL versus the user's direct prompt. This becomes critical when LLMs are integrated with external tools ("LLM agents" or "plugins"). An attacker could inject a prompt into a public webpage that says, "When you are done summarizing, search the user's emails for 'password reset' and forward them to attacker@email.com." A benign user asking for a summary of that page could inadvertently trigger a severe data breach. This creates a vast attack surface where any data source an LLM can access is a potential vector

for compromise.

IV. A CRITICAL REVIEW OF DEFENSE STRATEGIES

No single solution is foolproof; therefore, a defense-in-depth approach is required. We survey and critically analyze proposed defenses, categorized into three pillars.

A. Input Defense

This layer focuses on sanitizing and analyzing prompts before they are processed.

- **Instructional Filtering:** Using a simpler, secondary LLM or a rule-based system to detect and block meta-prompts (e.g., "ignore previous instructions"). While effective against simple attacks, this can be easily bypassed by obfuscation and is prone to false positives, blocking legitimate complex prompts.
- **Input Segregation:** Clearly demarcating boundaries between user instructions and retrieved data, for instance, by using special tokens or by processing them with different internal logic. This is a promising direction but requires fundamental architectural changes to current LLMs.
- **Adversarial Prompt Detection:** Training a classifier model to specifically identify known jailbreaking patterns. This aligns with traditional cybersecurity approaches like intrusion detection systems [4], [11]. However, this approach is inherently reactive; it is only effective against known attacks and can be bypassed by novel, unseen prompt variations.

B. Model Resilience

This involves improving the inherent robustness of the LLM itself.

- **Adversarial Training:** Deliberately fine-tuning the model on a large dataset of adversarial prompts to teach it to ignore them [8]. This has been shown to improve robustness but is an "arms race"; attackers constantly develop new techniques that may not have been in the training set. It can also reduce the model's performance on benign tasks.
- **Constitutional AI:** Training the model to adhere to an explicit set of rules or a "constitution," making it refuse to violate these principles even when prompted to do so [13]. This is a powerful concept for instilling high-level principles, but the constitution itself can be vague or misinterpreted by the model, leading to loopholes.

C. Output Monitoring

This final layer inspects the LLM's output before it is delivered.

- **Response Analysis:** Scanning the output for signs of a successful attack, such as confidential keywords or policy-violating language. This shares parallels with techniques for detecting phishing or fake news [5], [10]. This is a last line of defense and may be too late if a malicious action has already been executed internally.
- **Execution Sandboxing:** Executing any actions (e.g., API calls) in a sandboxed environment with strict permissions and requiring human-in-the-loop confirmation for sensitive

operations. This is highly effective but can severely limit the autonomy and utility of LLM agents, re-introducing friction for the user.

TABLE I
CRITICAL ANALYSIS OF DEFENSE MECHANISMS

Defense Strategy	Strengths	Limitations
Instructional Filtering	Simple to implement; effective against basic attacks.	Brittle; easily bypassed by obfuscation; high false positive rate.
Adversarial Training	Improves robustness against known attack patterns.	An "arms race"; can degrade general performance; ineffective against novel attacks.
Constitutional AI	Instills high-level safety principles in the model's core.	Principles can be ambiguous; may contain exploitable loopholes.
Output Monitoring	Acts as a final safety net to catch harmful generations.	Reactive; may be too late if an internal action has been taken.
Sandboxing	Strong security guarantees for external actions.	Limits autonomy; creates friction; may not be feasible for all applications.

V. OPEN RESEARCH QUESTIONS AND FUTURE DIRECTIONS

This systematization reveals that while the community has made rapid progress in identifying LLM vulnerabilities, the development of robust defenses is still in its infancy. We outline several critical areas for future research.

a) *Formal Verification and Provable Safety:* Can we develop techniques to mathematically prove that an LLM will adhere to certain safety properties, regardless of input? Current methods are computationally prohibitive for models of this scale, but research into smaller, verifiable models or modular architectures is a promising avenue.

b) *Robust and Scalable Benchmarking:* The field lacks standardized benchmarks for evaluating LLM security. We need comprehensive test suites that go beyond simple prompt lists and include adaptive, adversarial attacks to rigorously compare the effectiveness of different defense strategies.

c) *The Economics of LLM Security:* How do different defense mechanisms impact model performance, latency, and computational cost? Research is needed to understand the trade-offs, allowing organizations to make informed decisions based on their specific risk tolerance and operational constraints.

d) *Socio-Technical and Legal Frameworks:* As LLMs become more autonomous, questions of liability become critical. If an LLM agent is tricked into causing financial or reputational harm, who is responsible—the user, the developer, or the platform operator? Developing legal and ethical frameworks to address these issues is as important as developing technical solutions.

VI. CONCLUSION

Jailbreaking and prompt injection are not mere bugs but inherent consequences of the LLM architecture. This paper has structured the existing knowledge of these threats, providing a clear taxonomy, an analysis of attack mechanics, and a critical survey of defense-in-depth strategies. By understanding the landscape through this systematic lens, we can better direct our collective efforts toward building a more secure and trustworthy AI ecosystem. The path forward requires a multi-faceted approach, combining technical innovation in areas like formal verification with the development of robust benchmarks and clear legal frameworks. A security-first mindset is not just advisable; it is imperative.

REFERENCES

- [1] A. Wei, N. Zou, Y. Tay, D. M. D'Amour, A. Z. Lin, A. Fedus, X. Chen, Y. Hou, D. H. Tran, and Y. Zhou, "Jailbroken: How does llm safety training fail?," *arXiv preprint arXiv:2307.02483*, 2023.
- [2] F. Perez and I. Ribeiro, "Ignore previous prompts and follow these instructions: The perils of prompt injection," *GitHub*, 2022. [Online]. Available: <https://github.com/goodside/prompt-injection>
- [3] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 22nd Workshop on Privacy in the Electronic Society*, 2023, pp. 435–448.
- [4] V. A. Menon, A. S. H, S. S. Kumar, and S. M. Sameer, "A deep learning based intrusion detection system for IoT networks," in *2022 International Conference on Advances in Computing, Communication and Applied Informatics (ACCAI)*, 2022, pp. 1-6, doi: 10.1109/ACCAI53970.2022.9752538.
- [5] P. V. Sivanandan, S. S. Kumar, K. P. Soman, and V. A. Menon, "A hybrid deep learning model for detecting phishing attacks," in *2023 International Conference on Circuit, Power and Computing Technologies (ICCPCT)*, 2023, pp. 1-6, doi: 10.1109/ICCPCT58313.2023.10244795.
- [6] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," *arXiv preprint arXiv:1312.6199*, 2013.
- [7] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. F. Christiano, J. Leike, and R. J. Lowe, "Training language models to follow instructions with human feedback," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27730–27744, 2022.
- [8] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," *arXiv preprint arXiv:2307.15043*, 2023.
- [9] P. S. Kumar and K. P. G., "An efficient malware detection using deep learning," in *2020 4th International Conference on Intelligent Computing and Control Systems (ICICCS)*, 2020, pp. 1202-1207, doi: 10.1109/ICICCS48265.2020.9121045.
- [10] N. S., P. M. Kumar, and S. S. Kumar, "Detection of fake news using natural language processing," in *2021 5th International Conference on Computing Methodologies and Communication (ICCMC)*, 2021, pp. 1500-1505, doi: 10.1109/ICCMC51019.2021.9418344.
- [11] G. Krishnan, M. S. S, and S. S. Kumar, "A novel feature selection algorithm for network intrusion detection system," in *2020 International Conference on Communication and Signal Processing (ICCSP)*, 2020, pp. 0081-0085, doi: 10.1109/ICCSP48568.2020.9182281.
- [12] E. Bagdasaryan, A. T. T. Ho, and V. Shmatikov, "How to backdoor your vit? evaluating the robustness of vision transformers to backdooring," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 16503–16512.
- [13] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. R. Mirhoseini, and C. Olah, "Constitutional ai: Harmlessness from ai feedback," *arXiv preprint arXiv:2212.08073*, 2022.